

Федеральное государственное бюджетное образовательное учреждение высшего
профессионального образования
«Санкт-Петербургский государственный политехнический университет»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

КУРСОВАЯ РАБОТА

по дисциплине «Объектно-ориентированное программирование»

на тему «Разработка телефонного справочника с использованием
библиотеки Qt»

Выполнил студент группы 5130203/40002

Чигринский З. Н.

Принял к.э.н., старший преподаватель ин.

_____ Йовановски Ненад

Оценка: _____

«__» _____ 20__ года

Санкт-Петербург

20__

Оглавление

1. ВВЕДЕНИЕ	3
2. ПОСТАНОВКА ЗАДАЧИ.....	3
2.1. Хранение данных	3
2.2. Валидация данных	3
2.3. Функциональность	3
2.4. Архитектура	3
3. РЕАЛИЗАЦИЯ.....	4
3.1. PhoneBook (Ядро приложения)	4
3.1.1. Класс Recording	4
3.1.1.1. Приватные поля.....	4
3.1.1.2. Конструкторы.....	4
3.1.1.3. Статические методы валидации (статичные)	4
3.1.1.4. Методы нормализации:	5
3.1.1.5. Сеттеры (публичные методы Set...)	5
3.1.1.6. Геттеры (публичные методы Get...)	5
3.1.1.7. Методы работы с телефонными номерами	5
3.1.1.8. Методы ввода/вывода:	5
3.1.2. Класс PhoneBook	5
3.1.2.1. Приватные поля.....	5
3.1.2.2. Конструкторы и деструктор	5
3.1.2.3. Методы управления записями:	6
3.1.2.4. Методы поиска:	6
3.1.2.5. Метод работы с файлами	6
3.1.2.6. Метод вывода данных в консоль	6
3.2. QtPhoneBook (Графический интерфейс)	6
3.2.1. Класс PhoneBookManager	6
3.2.1.1. Данные	6
3.2.1.2. Методы управления интерфейсом	7
3.2.1.3. Методы работы с данными	7
3.2.1.4. Методы работы с базой данных	7
3.2.1.5. Точка входа в GUI	7
3.2.2. Вспомогательный класс QVListWidget	8
3.2.2.1. Функциональность	8
3.2.3. Вспомогательные функции	8
4. Тестирование	8
5. Заключение.....	16

1. ВВЕДЕНИЕ

Целью данного проекта была разработка кроссплатформенного приложения «Телефонный справочник» с использованием библиотеки Qt. Приложение позволяет управлять контактами: добавлять, удалять, редактировать записи, выполнять поиск и сортировку, а также сохранять данные в текстовый файл и базу данных PostgreSQL. Проект выполнялся поэтапно: сначала была создана консольная версия с сохранением в файл, затем добавлен графический интерфейс, реализована работа с базой данных и оптимизация операций с объектами.

2. ПОСТАНОВКА ЗАДАЧИ

Требовалось разработать приложение, удовлетворяющее следующим условиям:

2.1. Хранение данных

- Обязательные поля для хранения: Имя, Фамилия, Отчество, адрес, дата рождения, email, телефонные номера (рабочий, домашний, служебный) в любом количестве;
- Обязательные поля для создания контакта: Имя, Фамилия, email, 1 номер;
- Поддержка хранения в текстовом файле и в базе данных PostgreSQL одновременно.

2.2. Валидация данных

- Проверка корректности вводимых данных с помощью регулярных выражений;
- Фамилия, имя, отчество — только буквы, цифры, дефис и пробел, должны начинаться с буквы и не заканчиваться дефисом;
- Телефон в форматах: +78121234567, 88121234567, +7(812)1234567 и т.д.
- Дата рождения — корректная дата, меньше текущей;
- Email — корректный адрес, состоящий из латинских букв, цифр и символа '@'.

2.3. Функциональность

- Добавление, удаление, редактирование записей;
- Поиск по нескольким полям;
- Сортировка по любому полю;
- Отображение данных в табличном виде.

2.4. Архитектура

- Чёткое разделение на классы (модель, представление, контроллер);
- Поддержка консольного и графического интерфейсов;
- Возможность расширения хранилищ (файл, БД).

3. РЕАЛИЗАЦИЯ

Проект имеет чёткое разделение на две логические части: **ядро приложения (PhoneBook)**, отвечающее за хранение данных и бизнес-логику, и **графический интерфейс (QtPhoneBook)**, построенный на библиотеке Qt, который обеспечивает взаимодействие с пользователем. Такая архитектура обеспечивает гибкость и возможность повторного использования кода.

3.1. PhoneBook (Ядро приложения)

Ядро представляет собой независимую от интерфейса библиотеку классов, реализующую всю логику работы с данными телефонной книги.

3.1.1. Класс Recording

Этот класс является моделью одной записи (контакта) в справочнике. Он инкапсулирует данные и логику их проверки.

3.1.1.1. Приватные поля

- Строка `firstName` – имя;
- Строка `lastName` – фамилия;
- Строка `middleName` – отчество;
- Строка `address` – адрес проживания;
- Строка `dateOfBirth` – дата рождения;
- Строка `email` – адрес электронной почты;
- Список строк `phoneNumbers` – список телефонных номеров.

3.1.1.2. Конструкторы

- Конструктор по умолчанию инициализирует пустую запись.
- Конструктор с параметрами (`firstName`, `lastName`, `email`, `phoneNumber`) создаёт запись с обязательными полями, автоматически вызывая нормализацию и проверку имён.
- Конструктор копирования создаёт глубокую копию существующей записи.

3.1.1.3. Статические методы валидации (статичные). Данные методы не требуют создания объекта и могут быть использованы для проверки данных извне.

- `CheckName(string)` – Проверяет строку на соответствие правилам для ФИО (буквы, цифры, дефис, пробел; начинается с буквы; не начинается и не заканчивается дефисом).
- `CheckDateOfBirth(string)` – Валидирует строку с датой, проверяя формат "ДД.ММ.ГГГГ", корректность чисел, количество дней в месяце и високосные года. Сравнивает дату с текущей.

- CheckEmail(string email, string first_name) – Проверяет наличие символа '@', базовую структуру email, а также наличие имени в самом адресе.
- CheckPhoneNumber(string) – Проверяет строку на соответствие одному из заданных форматов телефонных номеров, анализируя последовательность цифр, скобок и дефисов.

3.1.1.4. Методы нормализации:

- Приватный метод NormalizeName() удаляет незначащие пробелы в начале и конце строки. Вызывается автоматически перед установкой имён.

3.1.1.5. Сеттеры (публичные методы Set...). Методы для установки значений полей. Каждый сеттер перед присвоением значения вызывает соответствующую проверку (см. п. 3.1.1.3.) и нормализацию (см. п. 3.1.1.4.). Если проверка не пройдена, значение не изменяется.

3.1.1.6. Геттеры (публичные методы Get...). Позволяют получить значения всех полей, включая список номеров телефонов.

3.1.1.7. Методы работы с телефонными номерами:

- AddPhoneNumber(string) – Добавляет новый номер в список после его валидации и проверки на дубликат;
- RemovePhoneNumber(string) – Удаляет конкретный номер из списка;
- DoesContainPhoneNumber(string) – Проверяет наличие номера в списке. Возвращает true, если номер содержится. Иначе возвращает ложь;
- RemoveEveryPhoneNumber() – Очищает весь список номеров.

3.1.1.8. Методы ввода/вывода:

- Write(string path): Записывает все данные записи в текстовый файл по указанному пути. Пустые поля заменяются на "_". Каждое поле и номер телефона записываются с новой строки, записи разделяются пустой строкой.
- Print(): Выводит отформатированную информацию о контакте в консоль (используется в консольной версии).

3.1.2. Класс PhoneBook

Этот класс является менеджером коллекции записей. Он управляет списком указателей на объекты Recording.

3.1.2.1. Приватные поля – Список указателей на Recording (list<Recording*> Recordings).

3.1.2.2. Конструкторы и деструктор:

- Конструктор по умолчанию создаёт пустую книгу;
- Конструктор копирования создаёт глубокую копию книги и всех её записей;

- Конструктор PhoneBook(string path) загружает данные из текстового файла, читает его построчно, создаёт объекты Recording и добавляет их в книгу;
- Деструктор проходит по списку и освобождает память, выделенную под каждый объект Recording.

3.1.2.3. Методы управления записями:

- AddRecording(Recording&) – Добавляет переданную запись в список.
- RemoveRecording(Recording*) – Удаляет конкретную запись из списка по указателю.
- GetAllRecordings() – Возвращает константный список всех записей для отображения.

3.1.2.4. Методы поиска:

- GetRecording(SEARCHMODE, string) – Осуществляет поиск по одному конкретному полю (например, только по фамилии или email).
- GetRecording(string firstName, ... string phoneNumber) – Гибкий поиск по комбинации любых полей. Поля, по которым поиск не требуется, помечаются символом "_". Возвращает первую найденную запись, удовлетворяющую всем заданным критериям.
- GetRecording(int index) – Возвращает запись по её порядковому номеру в списке.

3.1.2.5. Метод работы с файлами – Rewrite(string path) – Полностью перезаписывает указанный файл, сохраняя в него все текущие записи книги в формате, используемом методом Recording::Write().

3.1.2.6. Метод вывода данных в консоль – Print() – выводит все текущие записи книги, поочерёдно вызывая функцию Recording::Print().

3.2. QtPhoneBook (Графический интерфейс)

Эта часть отвечает за визуальное представление и взаимодействие с пользователем. Она построена на основе виджетов Qt и использует ядро PhoneBook для операций с данными.

3.2.1. Класс PhoneBookManager

Главный управляющий класс GUI. Он связывает интерфейс с логикой ядра.

3.2.1.1. Данные:

- CurrentBook, CurrentRecording – Текущие активные объекты ядра для работы.
- DataBase – Объект для подключения к PostgreSQL.
- Recordings_table, table_view – Модель (QStandardItemModel) и представление (QTableView) для отображения данных в таблице.

- `stacked_widget` – Виджет (`QStackedWidget`) для переключения между страницами интерфейса.
- `lineEdits`, `phoneEdits` – Массив полей ввода и кастомный виджет для редактирования данных записи.

3.2.1.2. Методы управления интерфейсом:

- `UpdateTablet()` – Полностью обновляет содержимое таблицы на основе данных из `CurrentBook`. Преобразует каждую запись `Recording` в строку таблицы.
- `FilterTablet(fieldStruct)` – Скрывает строки таблицы, не соответствующие критериям поиска, введённым в полях фильтра. Поиск осуществляется по подстроке.
- `UpdateEditing()` – Заполняет форму редактирования данными из `CurrentRecording` или очищает её для создания новой записи.
- `ChangePage(PAGE)` – Переключает текущую страницу `stacked_widget` (Главное меню, Таблица, Загрузка, Сохранение, Редактирование). В зависимости от страницы может выполнять подготовительные действия (например, сброс текущих данных).

3.2.1.3. Методы работы с данными:

- `BuildRecording(fieldStruct)` – Создаёт и возвращает новый объект `Recording` на основе данных, введённых в форме.
- `GetRecordingFromTableView()` – Определяет, какая запись в таблице выбрана пользователем, и находит соответствующий ей объект `Recording` в `CurrentBook`.
- `CheckRecordingFields(fieldStruct)` – Вызывает статические методы валидации класса `Recording` для проверки корректности всех полей перед сохранением.

3.2.1.4. Методы работы с базой данных:

- `ConnectToBD()`, `DisconnectWithBD()` – Устанавливают и закрывают соединение с СУБД PostgreSQL.
- `WriteToBD(QString tablet)` – Создаёт в БД таблицу с заданным именем, если её нет, и записывает в неё все записи из `CurrentBook`. Преобразует список телефонов в формат массива PostgreSQL.
- `ReadFromBD(QString tablet)` – Читает все записи из указанной таблицы БД, создаёт для них объекты `Recording` и загружает в новый объект `PhoneBook`.

3.2.1.5. Точка входа в GUI – Метод `Start()` является основной функцией, запускающей приложение. В нём:

- Создаётся приложение (`QApplication`) и главное окно (`QWidget`);

- Инициализируются все страницы интерфейса с размещением кнопок, полей ввода, таблицы;
- Настраиваются сигналы и слоты;
- Запускается основной цикл обработки событий Qt (app.exec()).

3.2.2. Вспомогательный класс QListWidget

Кастомный виджет, унаследованный от QVBoxLayout, для динамического управления списком текстовых полей (в данной курсовой работе используется для ввода нескольких телефонных номеров).

3.2.2.1. Функциональность:

- AddElement() – Динамически добавляет на форму новую горизонтальную компоновку (QHBoxLayout), содержащую поле ввода (QLineEdit) и кнопку "Удалить".
- CopyElement(string) – Добавляет новое поле и сразу заполняет его переданным значением.
- DeleteElement() – Удаляет конкретный набор виджетов (поле и кнопку).
- DeleteEveryElement() – Очищает весь список полей.
- GetEveryElement() – Возвращает все введенные номера телефонов в виде одной строки, где номера разделены символом новой строки.

3.2.3. Вспомогательные функции

- QFileSavePhoneBook(), QFileReadPhoneBook() – Реализуют чтение и запись объекта PhoneBook в текстовый файл с использованием классов Qt (QFile, QTextStream). Аналогичны методам ядра, но интегрированы в Qt-приложение.
- ConvertStringToDate(), pgArrayToStringList() – Утилиты для преобразования строкового формата даты в объект QDate и для парсинга массива строк из PostgreSQL в QStringList.

4. Тестирование

Данная глава посвящена тестированию приложения.

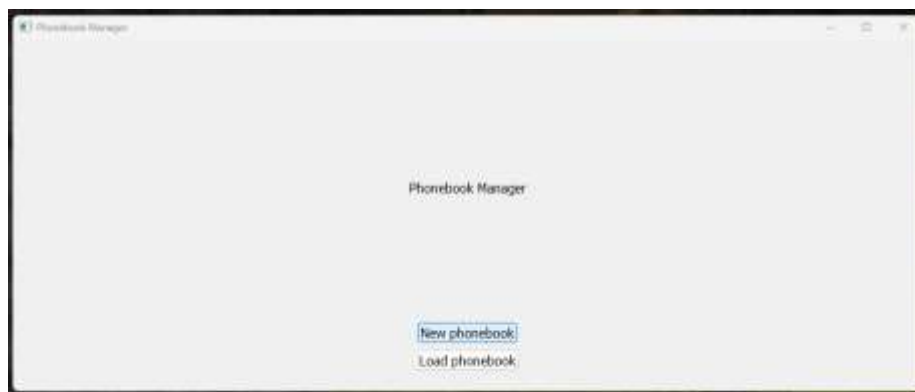


Рис. 4.1. Начальный экран

На начальном экране (Рис. 4.1) расположены две кнопки: “New phonebook” – для создания новой телефонной книжки и “Load phonebook” – для импортирования телефонной книжки.



Рис. 4.2. Пустая таблица

На экране с таблицей находится непосредственно таблица с записями (Рис. 4.2) (пока пустая, так как была создана новая телефонная книжка). Над таблицей находятся 7 кнопок сортировки – каждая соответствует своему полю. Над этими кнопками также находятся 7 полей ввода для фильтрации по соответствующим полям.

Слева от таблицы находятся кнопки “Add”, “Change”, “Remove”, “Save phonebook” и “Exit without saving” (двигаясь сверху вниз).

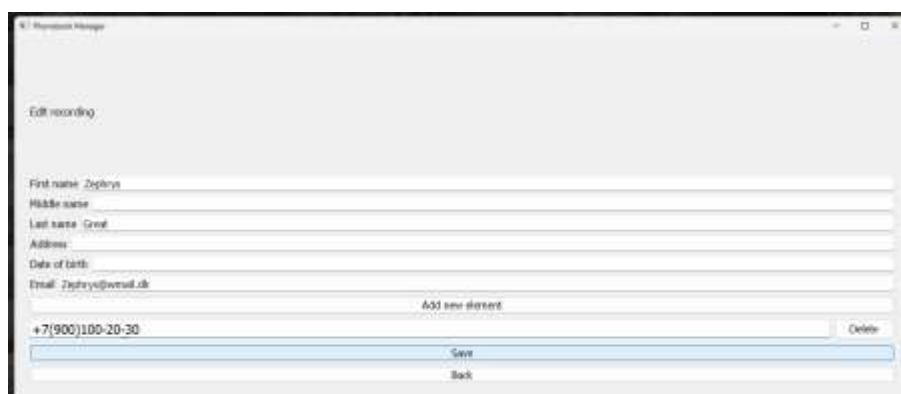


Рис. 4.3. Меню создания и редактирования записей

Кнопка “Add” открывает меню создания и редактирования записей (Рис. 4.3.). Чтобы создать новую запись необходимо ввести корректную информацию в поля “First name”, “Last name”, “Email” и добавить минимум 1 подходящий телефонный номер. Если информация в полях будет некорректна, то программа выведет предупреждение “Error: incorrect fields” и предложит исправить поля.

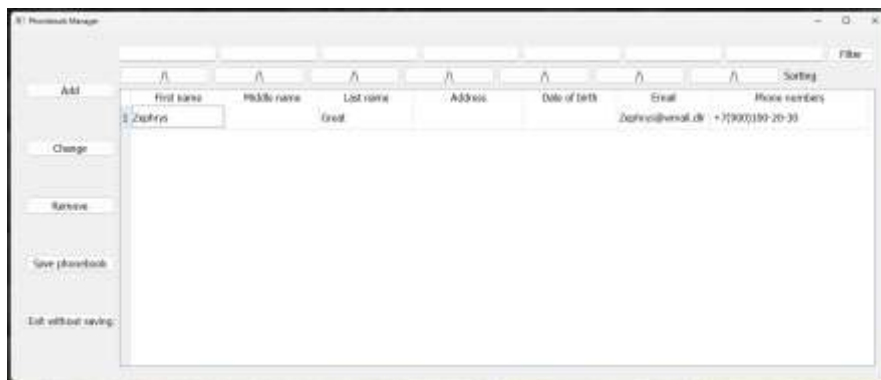


Рис. 4.4. Таблица с одной записью

Если в полях нет некорректной информации, то при нажатии кнопки “Save” снова открывается таблица с записью (Рис.4.4). Вся информация была верно интерпретирована программой.

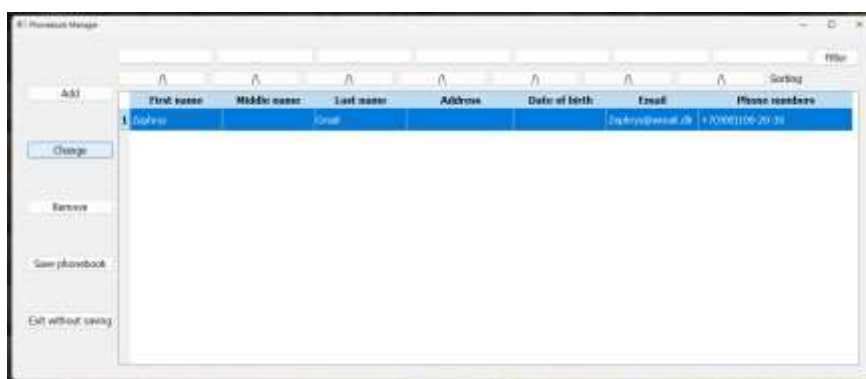


Рис. 4.5. Выбор записи для редактирования

Чтобы редактировать или удалить запись, необходимо выбрать строчку в таблице (Рис. 4.5) и затем нажать “Change” или “Remove”.

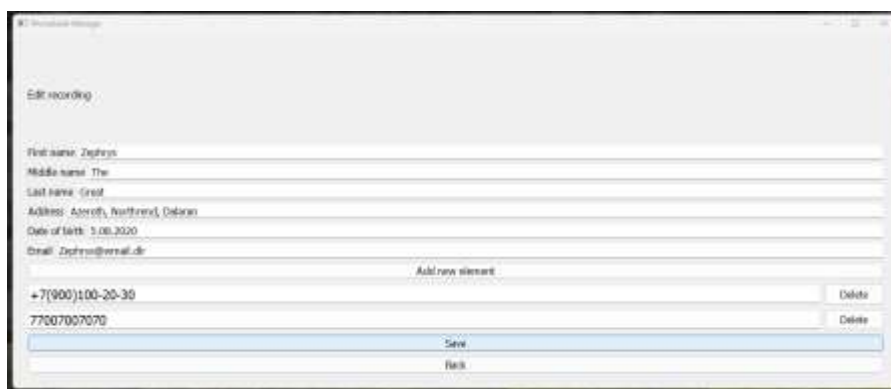


Рис. 4.6. Меню с отредактированной записью

Меню для редактирования записей (Рис. 4.6) выглядит так же, как и меню создания записей (Рис. 4.3).

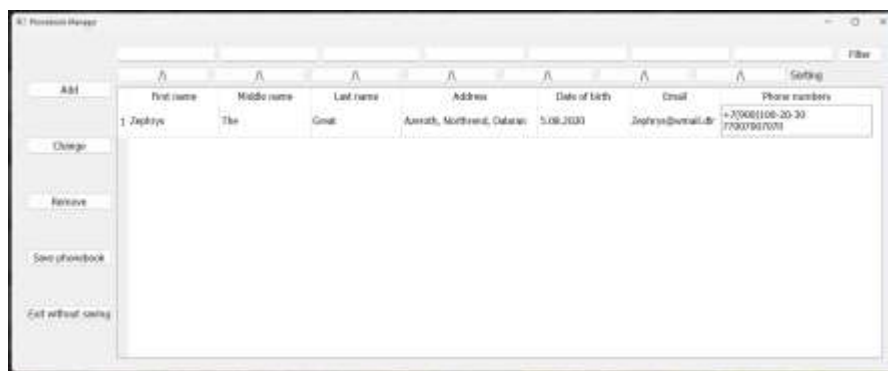


Рис. 4.7. Таблица с отредактированной записью

Как видно из таблицы (Рис. 4.7) информация о всех изменениях была верно интерпретирована программой.



Рис. 4.8. Таблица с тремя записями

Для теста следующих функций необходимо создать ещё несколько записей, представленных в таблице под номерами 2 и 3 (Рис. 4.8).



Рис. 4.9. Выбор записи для удаления

Удаление записей производится таким же образом, как и редактирование: необходимо выбрать запись для удаления, и нажать кнопку “Remove” (Рис.4.9).

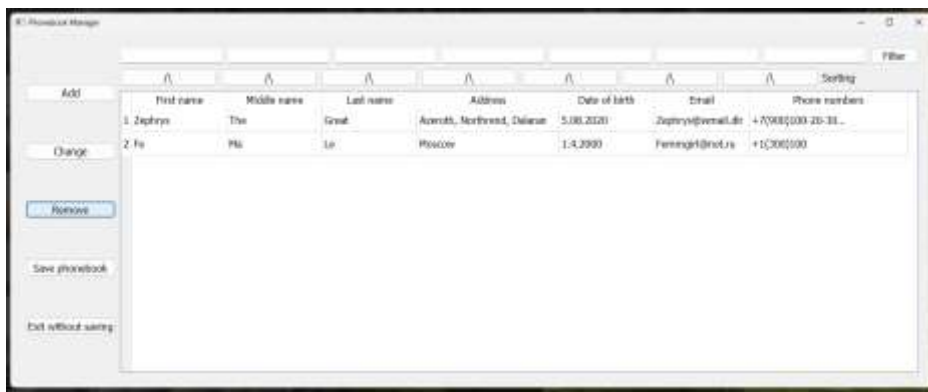


Рис. 4.10. Результат удаления записи

Как видно из таблицы (Рис. 4.10) информация о удалении третьей записи была верно интерпретирована программой.

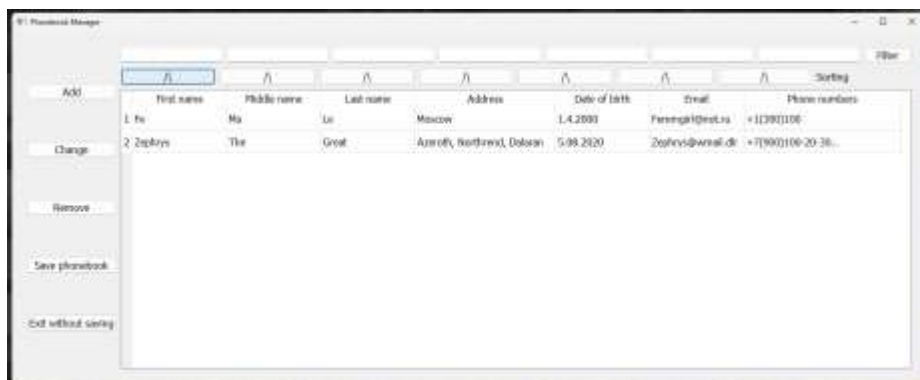


Рис. 4.11. Сортировка по имени (по убыванию)

Нажимая на кнопки с символами “^”, происходит сортировка данной таблицы по убыванию по соответствующему полю (Рис. 4.11). Чтобы сортировать по возрастанию, необходимо повторно нажать на кнопку.

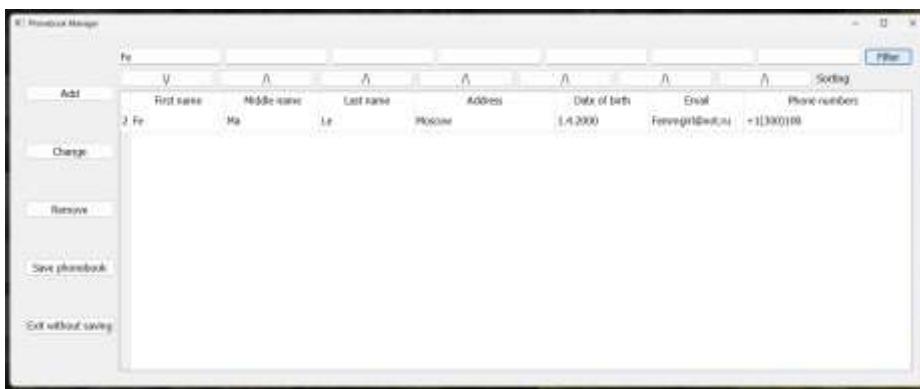


Рис. 4.12. Фильтрация по имени

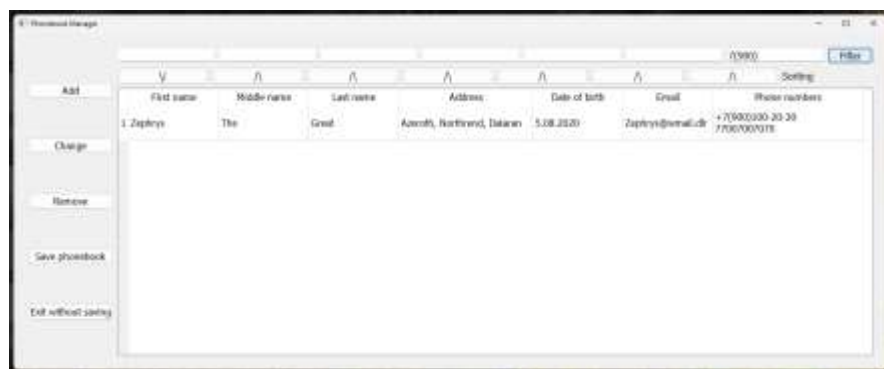


Рис. 4.13. Фильтрация по номерам телефонов

Верхние поля предназначены для фильтрации таблицы (Рис. 4.12-13).



Рис. 4.14. Меню сохранения таблицы

Кнопка “Save” на экране таблицы (Рис. 4.2) открывает окно с двумя зонами (Рис. 4.14). Верхняя зона позволяет сохранять таблицу в виде текстового файла, нижняя – в виде SQL-таблицы.



Рис. 4.15. Выбор названия сохраняемого файла

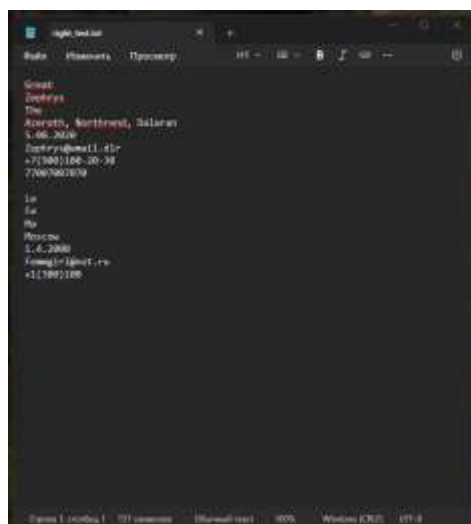


Рис. 4.16. Текстовый файл с данными таблицы

На Рис. 4.15 показано имя сохраняемого файла; а на Рис. 4.16 – результаты сохранения.



Рис. 4.17. Выбор названия сохраняемой таблицы

Query Query History

1 SELECT first_name, middle_name, last_name, address, email, birth_date, phones FROM right_test

Data Output Messages Notifications

Рис. 4.18. Сохранённая SQL-таблица

На Рис. 4.17 показано имя сохраняемой SQL-таблицы; а на Рис. 4.18 – результаты сохранения.

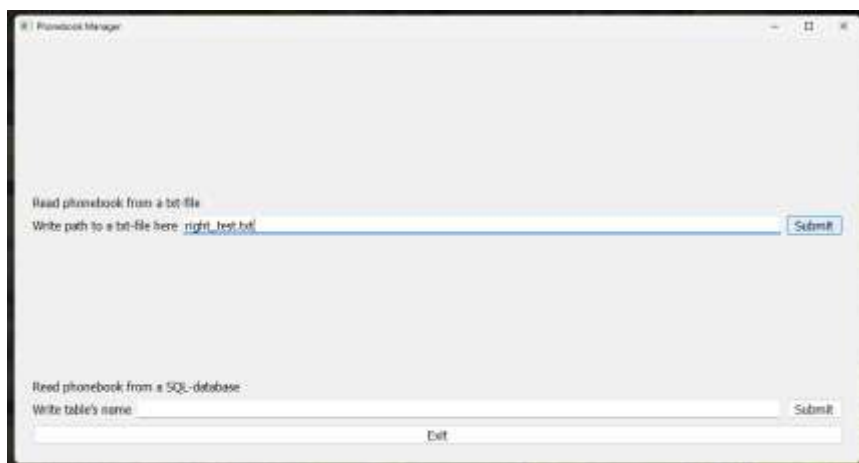


Рис. 4.19. Меню импорта из текстового файла

Кнопка “Load phonebook” на начальном экране (Рис. 4.1) открывает меню импорта из текстового файла и SQL-таблицы (Рис. 4.19).

	First name	Middle name	Last name	Address	Date of birth	Email	Phone numbers
1	Zephyrs	The	Great	Azeroth, Northrend, Dalaran	5.08.2020	Zephyrs@vmail.de	+7(900)100-20-...
2	Fa	Na	Le	Moscow	1.4.2000	Fennigki@not.ru	+1(300)100

Рис. 4.20. Полученная табличка из текстового файла

На Рис. 4.19 показано имя импортируемого файла; а на Рис. 4.20 – результаты импорта.



Рис. 4.21. Выбор названия импортируемой таблицы

	First name	Middle name	Last name	Address	Date of birth	Email	Phone numbers
1	Zapkov	Ivan	Grig	Adorfa, Northwind, Belarus	05.08.2000	Zapkov@emal.by	+37500100 23 20 77067007070
2	Ivan	Ivan	Grig	Moscow	01.04.2000	Ivan@emal.ru	+7000000

Рис. 4.22. Полученная табличка из SQL-таблицы

На Рис. 4.21 показано имя импортируемой SQL-таблицы; а на Рис. 4.22 – результаты импорта.

5. Заключение

В ходе выполнения проекта было разработано полнофункциональное приложение «Телефонный справочник», отвечающее всем поставленным требованиям. Были реализованы:

- Чёткая архитектура с разделением на логику и интерфейс.
- Корректная валидация данных с помощью регулярных выражений.
- Удобный графический интерфейс на Qt с табличным представлением, сортировкой и поиском.
- Поддержка нескольких хранилищ: текстовый файл и база данных PostgreSQL.
- Возможность добавления, редактирования, удаления и фильтрации записей.

Приложение успешно прошло тестирование на основных сценариях использования. Все функции работают корректно, интерфейс интуитивно понятен. Проект может быть расширен за счёт добавления новых видов хранилищ, улучшения дизайна или добавления дополнительных полей для контактов.